

Analysis: Why the GTO Agent Plays at –300 bb/100 Against Slumbot

Poker AI (ASI) v6 — gto_pure agent evaluation

July 2026

1. Problem Statement

The GTO solver plays at –50 bb/100 against Slumbot, while the trained gto_pure agent (no MCTS, no opponent embedding) plays at **–300 bb/100**. This 250 bb/100 gap must be explained by defects in the training pipeline, the inference path, or the evaluation setup.

Configuration: gto_pure modifier = {temperature: 0.5}, training data 150k scenarios (2–9 players, stacks 100–3000 BB), evaluation against Slumbot (heads-up, 200 BB deep, BB=100).

2. Bug: Missing Legal Action Mask in Modifiers

Severity: CRITICAL. Files: modifiers.py:169, sharded.py:341.

2.1 The data generation path (correct)

In generate.py (lines 996–999), action probabilities are computed with a legal action mask:

```
legal_mask_t = torch.tensor(meta["legal_mask"], dtype=torch.bool)
masked_evs = all_evs.masked_fill(~legal_mask_t, float("-inf"))
action_probs = F.softmax(masked_evs / normalizer, dim=0)
```

This correctly zeros out: (a) fold when checking is free (facing_bet=0), and (b) raise bins that collapse into the call or duplicate all-in.

2.2 The modifier path (BROKEN)

When apply_modifiers() or apply_modifier_single() recomputes action_probs from modified EVs, **the legal mask is not applied**:

```
evs_t = torch.tensor(evs, dtype=torch.float32)
s["action_probs"] = F.softmax(evs_t / normalizer, dim=0).tolist()
# <-- NO legal mask! All 14 actions get nonzero probability
```

The scenario dict stores action_evs (raw, unmasked EVs) but does NOT store the legal mask. When modifiers recompute probs from EVs, the mask information is lost.

2.3 Concrete impact

Fold when checking is free:

When facing_bet=0, fold EV = –hero_invested (typically –10 for BB). With normalizer = $\max(10+0, 10) \times 0.5 = 5$:

$$P(\text{fold}) = \text{softmax}(-10 / 5) = \text{softmax}(-2) \approx 0.01\text{--}0.05$$

The model learns to sometimes fold when it can check for free. This is **pure EV loss** — every such fold gives up the right to see a free card.

Raise bin duplication:

When K raise bins all collapse to all-in (because actual_bet ≥ stack), they all carry the same EV as the explicit all-in action. Without masking:

- K+1 actions share the all-in probability mass equally

- The model learns to distribute "go all-in" across $K+1$ indices
- At inference, bins are masked $\rightarrow$ the real all-in action (index 13) has undertrained logits $\rightarrow$ incorrect all-in frequency

This bug affects ALL agents with modifiers, including gto_pure (which has a temperature modifier that triggers the recomputation). Since every agent in the multi_agent config has modifiers, **100% of training data is corrupted**.

3. Double Temperature at Inference

Severity: HIGH.

3.1 Training targets

The GTO data generation computes action probabilities as:

```
target_probs = softmax(EVs / normalizer)
```

```
normalizer = max(pot + facing_bet, bb)  $\times$   $T_{\text{train}}$ 
```

For gto_pure, $T_{\text{train}} = 0.5$ (modifier overrides base 0.2).

3.2 What the network learns

The KL loss minimizes $\text{KL}(\text{target_probs} \parallel \text{softmax}(\text{logits}))$. If the network converges, then:

```
softmax(logits)  $\approx$  target_probs = softmax(EVs / normalizer)
```

```
 $\Rightarrow$  logits  $\approx$  EVs / normalizer + C
```

3.3 Inference applies temperature AGAIN

At inference (slumbot_eval.py:1172), the code divides logits by temperature from the checkpoint:

```
probs = F.softmax(logits / max(bundle["temperature"], 1e-3), dim=0)
# temperature = 0.5 (from checkpoint)
```

Substituting the learned logits:

```
softmax(logits /  $T_{\text{eval}}$ ) = softmax((EVs / normalizer + C) /  $T_{\text{eval}}$ )
= softmax(EVs / (normalizer  $\times$   $T_{\text{eval}}$ ))    (C/T is a constant shift,
softmax-invariant)
= softmax(EVs / (max(pot+fb, bb)  $\times$   $T_{\text{train}}$   $\times$   $T_{\text{eval}}$ ))
= softmax(EVs / (max(pot+fb, bb)  $\times$  0.5  $\times$  0.5))
= softmax(EVs / (max(pot+fb, bb)  $\times$  0.25))
```

The effective temperature is **0.25**, not 0.5. The solver at eval uses 0.2. So the agent is slightly softer ($0.25 > 0.2$) than the solver.

Critical secondary effect: if the network has NOT converged perfectly (which is certain with 150k samples), then the logits contain errors. Dividing by $T < 1$ **exponentially amplifies ranking errors**. Example: if the model incorrectly ranks action A above action B by Δ in logit space, dividing by 0.5 doubles the gap to 2Δ , and softmax magnifies this exponentially.

4. Multiway Training vs. Heads-Up Evaluation

Severity: HIGH.

Training data is generated with `num_players = random.randint(2, 9)` (generate.py:704). Only 12.5% of scenarios are heads-up. Slumbot is **strictly heads-up**.

Key strategic differences between 9-max and heads-up:

- **Opening ranges:** 9-max EP opens ~15% of hands; HU opens ~80%
- **Aggression:** HU requires far more bluffing and thin value bets
- **Position dynamics:** 2-player pot dynamics are fundamentally different from 5+ player pots
- **Stack-to-pot ratios:** HU pots stay smaller relative to stacks, requiring different bet sizing

The model has ~18k HU training samples out of 150k total (12.5%). For a transformer with `d_model=384` and 24+ layers, this is severely insufficient to learn HU-specific strategy. The model will default to multiway-biased play: tighter ranges, less bluffing, less aggression. Against Slumbot, this is heavily exploitable.

5. Soft Training Targets Reduce Signal

Severity: MEDIUM.

The `gto_pure` modifier sets `temperature=0.5`, while the base solver uses 0.2. The softmax temperature controls the "peakedness" of the training distribution.

For a typical decision point:

Action	EV (chips)	P (T=0.2)	P (T=0.5)
fold	-100	0.00	0.01
call	50	0.12	0.11
raise 0.5x	80	0.35	0.16
raise 1x	90	0.45	0.19
all-in	20	0.08	0.09

Table 1. Example: how temperature flattens the training distribution (illustrative values).

With `T=0.5`, the best action (raise 1x) gets only 19% instead of 45%. The model receives less informative gradient per sample. With limited training data (150k scenarios, 5 epochs for phases 1–2, 1 epoch for phase 3), this compounds with the data efficiency problem.

6. Insufficient Training Data for Model Size

Severity: MEDIUM.

The model has `d_model=384`, 6+6+6+6 attention layers (encoder, decoder, value, action), ~10M+ parameters. Training uses:

- 150k scenarios × ~10 decisions/hand ≈ 150k samples
- Phase 1–2: 5 epochs, `batch_size=64` ≈ 11.5k steps
- Phase 3 (joint): **1 epoch**, `batch_size=256` ≈ 585 steps

Phase 3 is particularly concerning: with just 585 gradient steps and three heads being trained jointly (perception + value + action), the model has very little opportunity to calibrate. The action head was tuned in phase 2 on frozen perception features, but phase 3 shifts the perception representation, requiring the action head to readjust—in just 585 steps.

7. Normalization Distribution Mismatch

Severity: LOW–MEDIUM.

Normalization statistics (`pot_mean`, `pot_std`, `stack_mean`, `stack_std`, `bets_mean`, `bets_std`) are computed from all training data (2–9 players, stacks 100–3000 BB). At Slumbot inference (2 players, 200BB deep):

- Pots are systematically smaller than the multiway training mean → z-score shifted negative
- Stacks are fixed at 200BB while training mean is ~500–1000BB → z-score shifted
- Bets from 2 positions + 7 zeros (padded to 9) vs. training with 2–9 actual bets

The model sees an out-of-distribution input regime at every Slumbot decision, potentially triggering behaviors learned for different game conditions.

8. Summary

#	Issue	Type	Est. Impact
1	Missing legal mask in modifiers (corrupted training targets)	Bug	50-150 bb/100
2	Double temperature at inference ($0.5 * 0.5 = 0.25$ effective)	Design	20-80 bb/100
3	Multiway training, HU evaluation (12.5% HU data for HU game)	Data	80-150 bb/100
4	Soft training targets ($T=0.5$) (reduced signal per sample)	Config	30-60 bb/100
5	Insufficient data for model size (150k samples, 1-epoch phase 3)	Scale	30-80 bb/100
6	Normalization mismatch (9-max stats for HU inference)	Data	20-40 bb/100

Table 2. Root causes of the 250 bb/100 gap, ranked by estimated impact.

These issues compound non-linearly. The legal mask bug poisons training targets, the multiway bias means the model never properly learns HU strategy, and the temperature issues further degrade the already-weak policy at inference. Together, they plausibly explain the full 250 bb/100 gap.

9. Recommended Fixes (Priority Order)

1. Fix legal mask in modifiers. Save the legal mask (or the information needed to reconstruct it) in each scenario dict. Apply it when recomputing action_probs in modifiers.py and sharded.py. This is a one-line fix per file if the mask is stored.

2. Remove double temperature at inference. Either: (a) use temperature=1.0 at inference (the training targets already incorporate the temperature), or (b) store the "intended inference temperature" separately from the training temperature and use it correctly. Option (a) is simplest and mathematically correct: if the network learned $\text{softmax}(\text{logits}) \approx \text{target_probs}$, then $\text{softmax}(\text{logits})$ is the right policy.

3. Increase HU representation in training data. Either: (a) train a separate HU agent for Slumbot evaluation, or (b) stratify num_players sampling to ensure at least 30–50% HU scenarios. Option (a) is recommended if Slumbot performance is a primary metric.

4. Use sharper training targets. Temperature=0.5 is too soft for 150k training samples. Use 0.2–0.3 for gto_pure (matching the solver).

5. Scale training data or reduce model size. Either increase to 500k–1M scenarios, or reduce d_model/layer counts for better sample efficiency.

Note: The 250 bb/100 gap is not a single catastrophic bug but a compound of 6 issues. The legal mask bug and the multiway data bias are the two largest contributors. Fixing just these two would likely bring the agent to –100–150 bb/100 range. Fixing all issues could bring it close to the solver's –50 bb/100.